Task 4 — BookStore Web API

User Stories

As a customer, I want to:

- Register an account and log in to the bookstore.
- Browse books and filter them by category, by author, or by price range.
- Place an order that contains one or more books.
- View my own past orders.

As an admin, I want to:

- Manage all books, categories, and authors.
- See all the orders placed by all customers.

As a frontend developer who will consume this API later, I want to:

- Have a clear and consistent API I can call from any client (web, mobile, or another service).
- Get clear error messages when something goes wrong.
- See documentation for every endpoint.

Requirements

- The API must be callable from a frontend running on a different origin.
- All write operations must require authentication.
- Some operations must only be allowed for admin users.
- Customers must only see their own orders. Admins can see all.
- Invalid requests must be rejected with a clear, structured error response.
- The API must never return internal database entities directly to the client.
- The API must never crash and expose internal error details. It should return a clean error response instead.
- Every endpoint must use the correct HTTP method and return the correct status code.

Tasks

1. Design the API endpoints for the bookstore. Decide the URL structure, the HTTP methods, the request bodies, and the response shapes.
2. Implement registration and login using JWT tokens.
3. Implement two roles: customer and admin. Restrict admin-only endpoints to admin users.

4. Implement the books endpoints: browse, search, filter, get details, create, update, delete.
5. Implement the categories and authors endpoints with full create, read, update, delete for admins.
6. Implement the orders endpoints: customers place orders, customers view their own orders, admins view all orders.
7. Validate all incoming requests and return clear, structured error messages on validation failure.
8. Never expose database entities to clients. Use separate objects for the data the API sends and receives.
9. Add pagination, searching, and filtering to the books listing endpoint.
10. Document the API with Swagger. The Swagger page must support testing authenticated endpoints.
11. Configure cross-origin requests so the API can be called from a frontend running on localhost.
12. Handle unexpected errors globally and return a clean response to the client.
13. Log important events such as login attempts, order creation, and errors.
14. Make sure the controllers do not contain business logic. Use dependency injection to separate concerns.

Submission Rules

- A working .NET 8 Web API project in a public GitHub repo.
- README must explain how to run the project, how to register the first admin, and how to test the API.